Train the agent, not just the model

Sergio Paniego | **@sergiopaniego** 🤗 Hugging Face

October 2026

THE VOCABULARY

An agent is a model plus a harness

Claude Code · Codex · OpenCode · Pi · Mini-SWE-Agent

🔁 runs the loop

it calls the model again and again, and decides when to stop

🛠️ owns the tools

it decides which tools exist, and it is the one that runs every call

📝 writes the context

system prompt, history, compaction: everything the model reads

🧩 reads the reply

in its own format, and rejects it if it does not fit

Same weights, different harness: LFM2.5-2.6B solves **62%** of the tasks in Mini-SWE-Agent and **33%** in Claude Code.

Source: The ultimate guide to multi-harness RL

THE MAP

How an agent gets trained in 2026

Post-training is where a pretrained model becomes an agent.

WHAT THE MODEL LEARNS FROM

imitate

SFT on agent traces

a demonstrator: the agent's past sessions

SFTTrainer

compress

Distillation

a teacher: its probabilities on every token

DistillationTrainer

reward

RL with verifiable rewards

a check you can run: did the tests pass?

GRPOTrainer

WHERE IT ACTS WHILE IT LEARNS

act

RL in environments

a sandbox with tools, in a loop the trainer runs

GRPOTrainer + OpenEnv

harness

RL through the harness

the real agent, running its own loop

AsyncGRPOTrainer + OpenEnv

The first three change the training signal, the last two move training into a loop where the model acts. **Mix and match: combine them, change the order, and skip what your data cannot support.**

THE STACK

Two open projects for every method

TRL

TRAINS THE MODEL

Hugging Face’s post-training library. One trainer per method, with the same shape for all of them.

- SFTTrainer
- DistillationTrainer
- GRPOTrainer
- AsyncGRPOTrainer

hf.co/docs/trl

OpenEnv

SERVES THE ENVIRONMENTS

One interface for RL environments, reset and step behind an HTTP server, each one a Space you can pull from the Hub.

- coding_env
- Harbor tasks
- browsers
- games
- simulators

and real coding agents inside them:

hf.co/docs/openenv

Both run on PyTorch.

IMITATE

SFT on agent traces

PRE-RECORDED TRACE (ONE WHOLE AGENT SESSION, SAVED AS DATA) — static dataset, nothing runs while training

1. USER fix the failing test in utils.py	2. AGENT read("utils.py")	3. TOOL def parse(s): ...	4. AGENT edit("utils.py", ...)
5. TOOL pytest: 1 failed	6. AGENT edit("utils.py", ...)	7. TOOL pytest: 3 passed	8. AGENT Fixed the off-by-one in parse()

☑ in the loss ☐ context only

Messages, tool calls and tool results, in the chat format the model already reads.

With **assistant_only_loss** the model learns the agent's turns and only reads the rest.

Claude Code, Codex and Pi traces can be shared on the Hub, with a trace viewer.

> The model only gets as good as the traces it copies, and good traces rarely show how to recover from a mistake.

Source: Agent traces on the Hub (docs)

IMITATE

SFT in TRL

The traces become chat messages, with their tool calls, and **SFTTrainer** trains on them.

sft.py PYTHON

```python
from huggingface_hub import snapshot_download
from trl import SFTConfig, SFTTrainer

# raw Pi sessions (JSONL), as the agent recorded them
path = snapshot_download("badlogicgames/pi-mono", repo_type="dataset")

# Pi's own format -> chat messages, keeping tool_calls and tool turns
dataset = sessions_to_messages(path)

trainer = SFTTrainer(
    model="Qwen/Qwen3-0.6B",
    train_dataset=dataset,
    args=SFTConfig(assistant_only_loss=True),
)
trainer.train()
```

→ Only because pi-mono is raw Pi JSONL. Traces already in chat format load with **load_dataset** and skip this.

Training Agents #1: SFT SFT docs pi-mono traces Episode 1 scripts

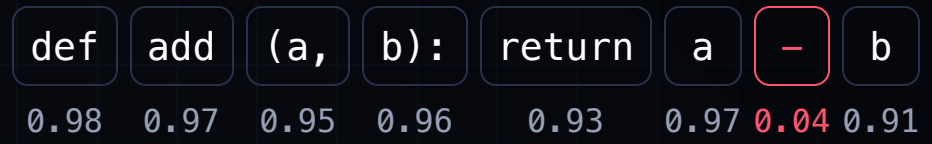
COMPRESS

Distillation: learn from a teacher

`return a _` next token? (illustrative)



ON-POLICY: THE TEACHER GRADES THE STUDENT'S TOKENS



The teacher puts 0.04 on −, so the student is corrected at the exact token where it went wrong.

A teacher gives a probability for every candidate token, where SFT gives only the right one.

On-policy, the student learns on text it wrote itself, so it sees its own mistakes.

The teacher: a bigger model, RL experts merged into one (MOPD), or **the model itself with a hint** (self-distillation).

> Teacher and student need the same tokenizer, and the student is limited by how good the teacher is.

Source: Distillation in 2026

COMPRESS

Distillation in TRL

The student generates from the prompts and the teacher scores every token. On-policy by default.

distill.py PYTHON

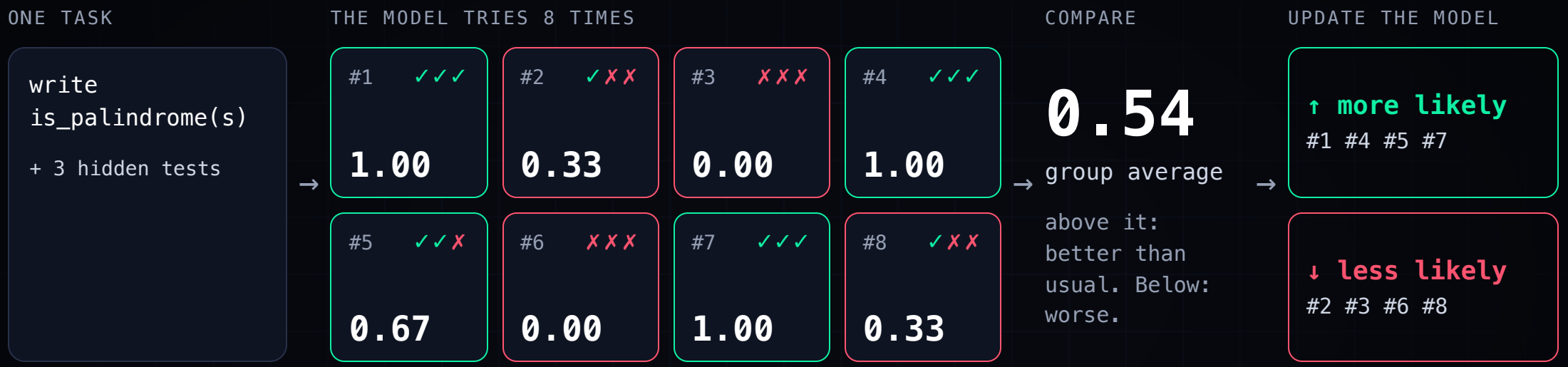
```python
from trl import DistillationTrainer

trainer = DistillationTrainer(
    model="Qwen/Qwen3-0.6B",          # student
    teacher_model="Qwen/Qwen3-4B",    # same tokenizer
    train_dataset=prompts,            # prompts only
)
trainer.train()
```

Training Agents #2: distillation Distillation in 2026 Distillation docs

REWARD

GRPO: learn from a check you can run



EACH ✓ IS ONE HIDDEN TEST PASSED, AND THE FRACTION PASSED IS THE REWARD (ILLUSTRATIVE NUMBERS)

The group is its own baseline, so there is no separate value model to train.

Tests, a compiler or a known answer all work as the reward.

Nobody shows the model the answer, so no demonstrator caps the result.

REWARD

GRPO in TRL

The reward is a plain Python function over the completions.

grpo.py PYTHON

```python
from trl import GRPOConfig, GRPOTrainer

def tests_pass(completions, test_list, **kwargs):
    # fraction of the task's asserts the code passes
    return [run_asserts(c[0]["content"], t)
            for c, t in zip(completions, test_list)]

trainer = GRPOTrainer(
    model="Qwen/Qwen3-0.6B",
    reward_funcs=tests_pass,
    train_dataset=mbpp,
    args=GRPOConfig(num_generations=8),  # the group
)
trainer.train()
```

Training Agents #3: RL Agentic RL in 2026 GRPO docs

ACT

RL inside an environment

GRPOTRAINER RUNS THIS LOOP, ONE SANDBOX PER ATTEMPT (ILLUSTRATIVE)



☐ model turns, trained on ☐ the OpenEnv sandbox answers

OpenEnv gives every environment one contract (reset, step), served as a Hub Space.

The reward moves into the sandbox, and comes after many turns of tool use.

ACT

Environments in TRL + OpenEnv

Same **GRPOTrainer** as before. The class's methods become tools, **get_reward** is the reward.

grpo_env.py PYTHON

```python
from trl import GRPOConfig, GRPOTrainer
from coding_env import CodeAction, CodingEnv  # OpenEnv client

class MBPPEnv:
    def reset(self, tests=None, **kwargs): ...  # fresh sandbox
    def get_reward(self) -> float: ...          # hidden tests
    def run_python(self, code: str) -> str:     # a tool
        r = self.client.step(CodeAction(code=code))
        return r.observation.stdout + r.observation.stderr

trainer = GRPOTrainer(
    model="Qwen/Qwen3-1.7B",
    train_dataset=mbpp_tasks,
    environment_factory=MBPPEnv,
    args=GRPOConfig(num_generations=8),
)
trainer.train()
```

HARNESS

Frontier labs now train coding models across several real harnesses

Kimi K3	builds its harnesses from modules, trains across them
Qwen3-Coder-Next	agentic coding data from six harnesses
MiMo-V2.6	a pool of task-adapted mini-harnesses
Poolside Laguna	1.3B tokens of OpenHands, OpenCode, Mini-SWE-Agent
Liquid LFM2.5	a harness picked at random per task

Source: The ultimate guide to multi-harness RL

HARNESS

Who drives the loop: the trainer or the harness

THE ENVIRONMENT WE JUST SAW

GRPOTrainer

runs the loop

↓ calls ↓ calls

model

the policy

tools

in the sandbox

✓ the trainer sees every turn

A HARNESS

Coding agent

runs its own loop

↓ calls ↓ calls

model

the policy

tools

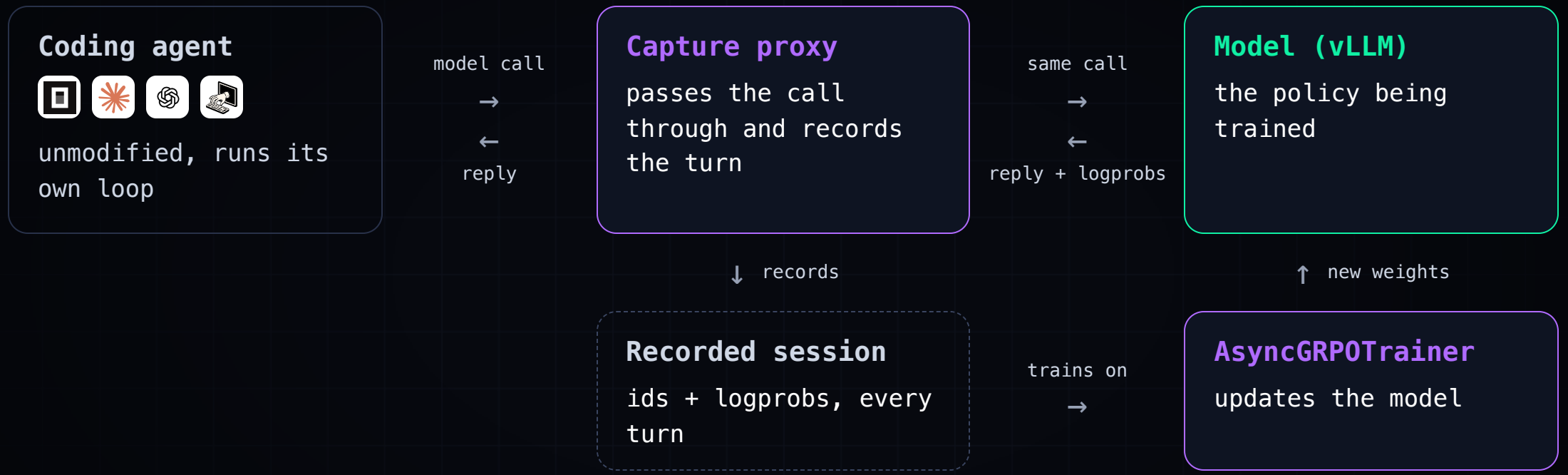
in the sandbox

✗ the trainer is not in the loop

The agent calls the model itself, so the trainer has to listen in on those calls.

HARNESS

The capture proxy



The harness talks to what it thinks is a normal model API. The proxy forwards each call to vLLM and records the token ids and logprobs of every turn, which is what GRPO needs to train.

HARNESS

In TRL + OpenEnv

multi_harness.py PYTHON

```python
from harbor_env import HarborEnv  # OpenEnv client
from trl.experimental.async_grpo import AsyncGRPOTrainer
from trl.experimental.async_grpo.openenv_harness import HarnessRolloutWorker

env = HarborEnv.from_hub(
    "FineEnvs/smoldataenv-multi-harness-harbor",  # tasks and sandboxes
    harness=random.choice(HARNESSES),  # picked per rollout
)

worker = HarnessRolloutWorker(
    harness_session_factory=env.session_factory,
    harness_adapter=None,  # the agent owns its loop
    rollout_reward_fn=reward,  # correctness and tool calls
)

AsyncGRPOTrainer(model=..., rollout_worker=worker).train()  # keeps generating while it trains
```

The harnesses come from Harbor, and OpenEnv runs them. Swapping the harness is **one argument**.

Simplified for the slide. Full training script Multi-harness RL

RECAP

Which one to reach for

You have traces of the agent you want	**SFT**
You have a stronger model with your tokenizer	**Distillation**
You can check the answer	**GRPO**
The task takes many steps with tools	**GRPO + OpenEnv**
Users run it inside a harness	**Train through the harness**

They combine. DeepSeek-V4 runs SFT then GRPO per domain, then merges the experts with on-policy distillation.

OVER TO YOU

Go deeper

TRAINING AGENTS, LIVE	READ	RUN	CODE
#1 SFT on agent traces	Distillation in 2026	SFT on tool calls	TRL
#2 Distillation	Agentic RL in 2026	Distillation	OpenEnv
#3 RL with GRPO	RL environments in 2026	GRPO + OpenEnv	TRL docs
#4 RL environments	Multi-harness RL	AsyncGRPO + Harbor	OpenEnv docs
The whole playlist	Agent glossary	Multi-harness models	Environments on the Hub

The deck is at hf.co/spaces/sergiopaniego/train-the-agent-slides

Sergio Paniego **@sergiopaniego** Hugging Face

THESE SLIDES